

AGATE – Best Practice Proposals

1. Ziel

Diese Seite beschreibt empfohlene Best Practices für die Strukturierung und Pflege von AGATE Tests.

Die Vorschläge sind nicht als starre technische Regeln zu verstehen. Sie sollen dabei helfen, Tests:

- lesbarer
- wartbarer
- wiederverwendbarer
- besser strukturiert
- einfacher debugbar

zu machen.

Der wichtigste Grundsatz lautet:

Business logic visible. Technical plumbing reusable.

Ein Testfall sollte primär beschreiben, **was fachlich passiert**. Wiederkehrende oder technisch komplexe Implementierungsdetails sollten hinter nativen AGATE Engines oder Reusable Modules gekapselt werden.

2. Haupttest auf Business-Ebene halten

Ein Haupttest sollte möglichst die fachliche Abfolge widerspiegeln.

Beispiel:

```
1 steps:
2
3   # Prepare test environment
4   - type: CALL
5     command: reusable.testsuite1.preparation.clean_database
6
7   # Upload input file
8   - type: OC
9     op: PUT
10    pod: "{E[env.ks.openShift.pod.name]}"
11    from: "{E[env.ks.testdata.basedir]}\input\change_prioritaeten.csv"
12    to: "{E[env.OMS.Report.TransferVerzeichnis]}/change_prioritaeten.csv"
13
14   # Execute business process
15   - type: CALL
```

```
16     command: reusable.testsuite1.business.change_priority
17
18     # Verify result
19     - type: CALL
20       command: reusable.testsuite1.validation.verify_result
21
```

Ein Tester sollte beim Lesen des Haupttests möglichst schnell erkennen können:

```
1 1. Testumgebung vorbereiten
2 2. Testdaten bereitstellen
3 3. Fachlichen Prozess ausführen
4 4. Ergebnis prüfen
5
```

Technische Details sollten den fachlichen Ablauf nicht unnötig verdecken.

3. Native AGATE Engines vor technischen Workarounds bevorzugen

Wenn AGATE eine bestimmte Operation bereits direkt unterstützt, sollte diese native Engine verwendet werden.

Beispiel:

Nicht bevorzugt:

```
1 - type: CMD
2   op: EXEC
3   command: |
4     powershell -Command ...
5     oc exec ...
6     oc cp ...
7
```

Bevorzugt:

```
1 - type: OC
2   op: PUT
3   pod: "${E[env.ks.openShift.pod.name]}"
4   from: "${E[env.ks.testdata.basedir]}\input.csv"
5   to: "/tmp/input.csv"
6
```

Vorteile:

- kürzere Tests
 - bessere Lesbarkeit
 - standardisiertes Fehlerhandling
 - zentrale Login-/Retry-Logik
 - weniger projektspezifische Shell-Kommandos
 - bessere Wartbarkeit
-

4. Reusable Modules haben zwei Aufgaben

Reusable Modules dienen nicht nur der klassischen Wiederverwendung.

Sie können zwei unterschiedliche Aufgaben erfüllen:

4.1. Technische Wiederverwendung

Beispiel:

```
1 shared.ssh_execute
2 shared.file_transfer
3 shared.database.execute_sql
4
```

Diese Module implementieren generische technische Funktionen und können von mehreren Applikationen verwendet werden.

4.2. Strukturierung eines Testfalls

Reusable Modules können außerdem verwendet werden, um einen komplexen Testfall in verständliche fachliche oder technische Abschnitte zu zerlegen.

Beispiel:

```
1 reusable.testsuite1.preparation.clean_database
2 reusable.testsuite1.business.change_priority
3 reusable.testsuite1.validation.verify_result
4
```

Diese Module müssen nicht zwingend von vielen verschiedenen Tests verwendet werden.

Ihr Mehrwert kann allein darin bestehen, den Haupttest:

- kürzer
- verständlicher
- besser strukturiert

zu machen.

5. Reusable Namespaces

Es wird vorgeschlagen, zwischen globalen und applikationslokalen Reusables zu unterscheiden.

5.1. `shared` – globale Reusables

Globale technische Reusable Modules liegen unter:

```
1 data/reusable/
2
```

Beispiel:

```

1 data/
2   └─ reusable/
3       ├── ssh_execute.yaml
4       ├── file_transfer.yaml
5       └─ database/
6           └─ execute_sql.yaml
7

```

Aufruf:

```

1 command: shared.ssh_execute
2

```

entspricht:

```

1 data/reusable/ssh_execute.yaml
2

```

Aufruf:

```

1 command: shared.database.execute_sql
2

```

entspricht:

```

1 data/reusable/database/execute_sql.yaml
2

```

`shared` sollte ausschließlich für Komponenten verwendet werden, die tatsächlich applikationsübergreifend einsetzbar sind.

Typische Beispiele:

```

1 shared.ssh_execute
2 shared.file_transfer
3 shared.database.execute_sql
4 shared.openshift.wait_for_pod
5

```

6. reusable – applikationslokale Reusables

Applikationsspezifische Module liegen unter:

```

1 data/<app>/reusable/
2

```

Beispiel:

```

1 data/
2   └─ app1/
3       └─ reusable/
4           └─ prepare_database.yaml
5

```

Aufruf:

```

1 command: reusable.prepare_database

```

entspricht:

```
1 data/app1/reusable/prepare_database.yaml
2
```

Die bestehende Dot-Notation kann dabei direkt zur Ordnerstruktur verwendet werden.

Beispiel:

```
1 command: reusable.group2.reusablexx
2
```

entspricht:

```
1 data/app1/reusable/group2/reusablexx.yaml
2
```

7. Strukturierung nach Test Suite

Bei größeren Applikationen mit mehreren Test Suites sollte zusätzlich nach Test Suite strukturiert werden.

Empfohlen:

```
1 reusable.<testsuite>.<bereich>.<modul>
2
```

Beispiel:

```
1 reusable.testsuite1.preparation.clean_database
2 reusable.testsuite1.business.change_priority
3 reusable.testsuite1.validation.verify_result
4
5 reusable.testsuite2.preparation.clean_database
6 reusable.testsuite2.business.change_priority
7 reusable.testsuite2.validation.verify_result
8
```

Physische Struktur:

```
1 data/
2   └─ app1/
3       └─ reusable/
4           └─ testsuite1/
5               └─ preparation/
6                   └─ clean_database.yaml
7               └─ business/
8                   └─ change_priority.yaml
9               └─ validation/
10                  └─ verify_result.yaml
11
12           └─ testsuite2/
13               └─ preparation/
14                   └─ clean_database.yaml
15               └─ business/
16                   └─ change_priority.yaml
```

```
17      └─ validation/
18          └─ verify_result.yaml
19
```

Damit ist bereits aus dem Namen ersichtlich:

```
1 Applikation
2   -> Test Suite
3       -> Phase
4           -> konkrete Funktion
5
```

8. Applikationsweite gemeinsame Reusables

Nicht jedes applikationsspezifische Reusable gehört nur zu einer Test Suite.

Für Funktionen, die mehrere Test Suites derselben Applikation verwenden, kann ein **common** - Bereich verwendet werden.

Beispiel:

```
1 reusable.common.prepare_database
2 reusable.common.login
3 reusable.common.clean_transfer_directories
4
```

Physisch:

```
1 data/app1/reusable/common/
2
```

Damit ergeben sich drei Ebenen:

```
1 shared.*
2
```

Globale, applikationsübergreifende technische Reusables.

```
1 reusable.common.*
2
```

Gemeinsame Reusables innerhalb einer Applikation.

```
1 reusable.<testsuite>.*
2
```

Test-Suite-spezifische Reusables.

9. Empfohlene fachliche Gruppen

Für Test-Suite-spezifische Reusables können standardisierte Bereiche verwendet werden.

Zum Beispiel:

```
1 preparation
```

```
2 business
3 validation
4 cleanup
5
```

Beispiel:

```
1 reusable.order_change.preparation.prepare_test_data
2 reusable.order_change.business.change_priority
3 reusable.order_change.validation.verify_priority
4 reusable.order_change.cleanup.remove_test_data
5
```

Dadurch entsteht ein konsistenter Testaufbau.

10. Naming Best Practices

Reusable-Namen sollten die Absicht beschreiben.

Nicht empfohlen:

```
1 reusable.000_datatenbank_preparation_all_1_0
2 reusable.101_delete_dynamischdaten_1_0
3 reusable.helper2
4 reusable.test123
5
```

Bevorzugt:

```
1 reusable.common.prepare_database
2 reusable.order_change.preparation.clean_database
3 reusable.order_change.business.change_priority
4 reusable.order_change.validation.verify_result
5
```

Ein guter Name sollte möglichst beantworten:

Was macht dieser Reusable?

und nicht:

Woher stammt dieser Reusable historisch?

11. Technische Details nicht unnötig im Haupttest zeigen

Ein Haupttest sollte nicht mit Infrastrukturdetails überladen werden.

Zum Beispiel sollte ein Tester möglichst nicht 20 einzelne Schritte sehen für:

```
1 mkdir
2 rm
3 oc exec
4 oc cp
5 SQL cleanup
```

```
6 pod lookup
7 SSH command
8
```

wenn diese logisch eine einzige Aktion darstellen:

```
1 Prepare test environment
2
```

Dann ist ein Reusable sinnvoll:

```
1 - type: CALL
2   command: reusable.testsuite1.preparation.prepare_test_environment
3
```

Die technische Implementierung bleibt im Reusable erhalten.

12. Reusables nicht zu groß machen

Die Kapselung sollte nicht dazu führen, dass ein komplettes Test-Szenario in einem einzigen Reusable verschwindet.

Nicht empfohlen:

```
1 - type: CALL
2   command: reusable.execute_complete_test
3
```

wenn dieser Reusable intern:

```
1 prepare
2 execute
3 validate
4 cleanup
5
```

enthält.

Dann ist im Haupttest die fachliche Struktur nicht mehr sichtbar.

Besser:

```
1 - type: CALL
2   command: reusable.testsuite1.preparation.prepare_environment
3
4 - type: CALL
5   command: reusable.testsuite1.business.execute_change
6
7 - type: CALL
8   command: reusable.testsuite1.validation.verify_change
9
```

Der Haupttest sollte die fachlich wichtigen Phasen weiterhin zeigen.

13. Faustregel für Reusable Modules

Ein Reusable ist sinnvoll, wenn mindestens einer der folgenden Punkte gilt:

- dieselbe Logik wird mehrfach verwendet
- die Implementierung ist technisch komplex
- mehrere technische Schritte bilden logisch eine einzige Aktion
- der Haupttest wird dadurch deutlich verständlicher
- die Logik soll unabhängig getestet oder gepflegt werden
- technische Änderungen sollen nicht alle Tests beeinflussen

Ein Reusable ist dagegen nicht automatisch sinnvoll, nur weil mehrere Schritte vorhanden sind.

14. Logging

Technische Reusable Modules sollten im normalen Testlauf möglichst wenig Log-Noise erzeugen.

Standard:

```
1 - type: CALL
2   command: reusable.testsuite1.preparation.prepare_environment
3
```

Technische Details werden nicht vollständig ausgegeben.

Für Debugging:

```
1 - type: CALL
2   command: reusable.testsuite1.preparation.prepare_environment
3   parameters:
4     verbose: "true"
5
```

Damit bleibt der normale Testlauf übersichtlich, während bei Problemen weiterhin alle technischen Details sichtbar gemacht werden können.

15. Business Flow im Log sichtbar halten

Auch der Test-Output sollte möglichst den fachlichen Ablauf zeigen.

Ideal:

```
1 Prepare test environment ..... SUCCESS
2 Upload input file ..... SUCCESS
3 Execute priority change ..... SUCCESS
4 Verify changed priority ..... SUCCESS
5
```

und nicht primär:

```
1 mkdir ...
```

```

2 rm ...
3 oc exec ...
4 oc cp ...
5 psql ...
6

```

Die technischen Details sollten nur bei Bedarf sichtbar werden.

16. Beispiel einer empfohlenen Struktur

```

1 data/
2   ├── reusable/
3     ├── ssh_execute.yaml
4     ├── file_transfer.yaml
5     └── database/
6         └── execute_sql.yaml
7
8   └── kams/
9       ├── T_change_priority.yaml
10
11       └── reusable/
12           ├── common/
13               ├── prepare_database.yaml
14               └── login.yaml
15
16           ├── change_priority/
17               ├── preparation/
18                   ├── clean_database.yaml
19                   └── prepare_filesystem.yaml
20
21               ├── business/
22                   └── change_priority.yaml
23
24               └── validation/
25                   └── verify_result.yaml
26

```

Verwendung:

```

1 steps:
2
3   - type: CALL
4     command: reusable.change_priority.preparation.clean_database
5
6   - type: CALL
7     command: reusable.change_priority.preparation.prepare_filesystem
8
9   - type: OC
10    op: PUT
11    pod: "{E[env.ks.openShift.pod.name]}"
12    from: "{E[env.ks.testdata.basedir]}\change_prioritaeten.csv"
13    to: "{E[env.OMS.Report.TransferVerzeichnis]}/change_prioritaeten.csv"
14
15   - type: CALL
16     command: reusable.change_priority.business.change_priority
17
18   - type: CALL
19     command: reusable.change_priority.validation.verify_result
20

```

Ein Tester kann den Ablauf damit verstehen, ohne die technische Implementierung jedes einzelnen Schrittes kennen zu müssen.

17. Zusammenfassung der vorgeschlagenen Namespaces

Namespace	Zweck	Beispiel
<code>shared.*</code>	globale, applikationsübergreifende technische Funktionen	<code>shared.ssh_execute</code>
<code>reusable.common.*</code>	gemeinsame Funktionen innerhalb einer Applikation	<code>reusable.common.prepare_database</code>
<code>reusable.<testsuite>.preparation.*</code>	Vorbereitung einer Test Suite	<code>reusable.order_change.preparation.clean_database</code>
<code>reusable.<testsuite>.business.*</code>	fachliche Aktionen	<code>reusable.order_change.business.change_priority</code>
<code>reusable.<testsuite>.validation.*</code>	Ergebnisprüfung	<code>reusable.order_change.validation.verify_result</code>
<code>reusable.<testsuite>.cleanup.*</code>	optionale Nachbereitung	<code>reusable.order_change.cleanup.remove_test_data</code>

18. Kernaussage

AGATE Tests sollten nicht danach optimiert werden, möglichst wenige Dateien zu besitzen.

Sie sollten danach optimiert werden, dass ein Tester den fachlichen Ablauf schnell versteht.

Deshalb:

Keep the business flow visible. Hide repeatable technical complexity behind native engines and reusable modules.

Oder noch kürzer:

Business logic visible. Technical plumbing reusable.